```c
#include <stdio.h>
```

Основы языка C

Стандарт, типы, операции и неопределенное поведение

ОБСУЖДЕНИЕ

Программирование и ИИ

01 Зачем учиться программировать, если код может написать ИИ?

02 Где ИИ уже ошибался, когда вы им пользовались?

03 Как понять, что сгенерированный код правильный?

Зачем программировать, если есть ИИ

1 **За код отвечаете вы, а не ИИ**

Чтобы проверить и исправить результат, нужно понимать код.

2 **ИИ ошибается уверенно**

Код выглядит правдоподобно, но работает неверно. В C это часто UB.

3 **Задачу надо уметь поставить**

Точная постановка задачи – уже половина программирования.

4 **Программирование учит мышлению**

Шаги, крайние случаи, рассуждения о корректности.

5 **ИИ усиливает тех, кто разбирается**

С ИИ специалист работает быстрее, а новичок быстрее ошибается.

ОБСУЖДЕНИЕ

Почему C

01 Зачем изучать язык 1970-х, если есть Python, Go и Rust?

02 Где C используется сегодня?

03 Что в C может оказаться сложнее, чем в Python?

Почему язык C

Понять, как устроены **программы и компьютер**

Увидеть, какие проблемы C **решают другие языки**

Маленькое ядро языка – **фокус на алгоритмах**

Подготовка к **C++, ассемблеру** и системному программированию

C в индустрии

```
ядро Linux
Windows
PostgreSQL
SQLite
CPython
OpenSSL
KasperskyOS
```

Почему C такой

C создавали в 1970-х, чтобы писать **Unix для разных машин** без ассемблера.

Близко к железу

Код почти напрямую превращается в машинные команды.

Без проверок

Границы массивов и переполнения не проверяются. Отсюда UB.

Разные машины

Размеры типов отличаются. Поэтому нужен стандарт.

`1972 Bell Labs · 1978 K&R · 1989 ANSI C · 1999 C99 · 2011 C11 · 2018 C17 · 2024 C23`

Стандарт языка C

Задает **синтаксис, семантику** и стандартную библиотеку.

Нужен для **переносимости**: программист и авторы компилятора опираются на общие правила.

У Python стандарта ISO нет: язык описывают документация и PEP.

Редакции: C99, C11, C17, C23.

ISO/IEC 9899

официальное название стандарта C

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

```
$ gcc -Wall -Wextra -Werror hello.c -o hello
$ ./hello
Hello, world!
```

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

Подключаем библиотеку ввода-вывода

```
$ gcc -Wall -Wextra -Werror hello.c -o hello
$ ./hello
Hello, world!
```

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

main – главная функция, точка входа

```
$ gcc -Wall -Wextra -Werror hello.c -o hello
$ ./hello
Hello, world!
```

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

printf выводит текст на экран

```
$ gcc -Wall -Wextra -Werror hello.c -o hello
$ ./hello
Hello, world!
```

Простейшая программа

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

return 0 – программа завершилась успешно

```
$ gcc -Wall -Wextra -Werror hello.c -o hello
$ ./hello
Hello, world!
```

Переменные

Переменная – именованное место в памяти. У нее есть **тип**, **имя** и **значение**.

```
int age = 18 ;
```

тип имя значение

```
double pi = 3.14;
char letter = 'A';
age = age + 1;     // значение можно менять
```

Типы данных

Тип задает **размер**, множество **значений**, представление и допустимые **операции**.

Целые

```
char
short
int
long
long long
```

signed и unsigned

Вещественные

```
float
double
long double
```

Логический

```
bool
true
false
```

из <stdbool.h>

Размеры типов: что гарантировано

```
sizeof(char) <= sizeof(short) <= sizeof(int)
             <= sizeof(long) <= sizeof(long long)
```

`sizeof(char) == 1`

Байт – CHAR_BIT бит, минимум 8

`int: от 16 бит`

Обычно 32, но это **не гарантия**

Как проверить

`sizeof`, `<limits.h>`

Точный размер

`<stdint.h>: int32_t`

ВОПРОС

Операция, выражение, оператор

Что здесь **операция**?

Что **выражение**?

Что **оператор**?

```
int result = 0;
result = 2 + 3 * 4;
printf("%d\n", result);
```

Операция, выражение, оператор

```c
int result = 0;
result = 2 + 3 * 4;
printf("%d\n", result);
```

Операции

operators

```
=   +   *
printf(...)
```

Выражения

expressions

```
2 + 3 * 4
result = 2 + 3 * 4
```

Операторы

statements

```
result = 2 + 3 * 4;
printf(...);
```

Выражение имеет **значение и тип**, оператор – **выполняется**.

Основные операции C

Арифметика

```
+   -   *   /   %   ++   --
```

Сравнения

```
<   <=   >   >=   ==   !=
```

Логика

```
!   &&   ||
```

0 – false, любое другое – true

Присваивание

```
=   +=   -=   *=   /=   %=
```

ВОПРОС

Целочисленная арифметика

```
  5 / 2     // ?
  5 % 2     // ?
 -5 / 2     // ?
 -5 % 2     // ?
```

Чему равно каждое выражение?

Целочисленная арифметика

```
 5 / 2    //  2
 5 % 2    //  1
-5 / 2    // -2
-5 % 2    // -1
```

Деление целых **отбрасывает дробную часть**: округление к нулю

Знак остатка **совпадает со знаком делимого**

Ленивая логика: && и ||

```
int a = 10;
int b = 0;

if (b != 0 && a / b > 2) {
    puts("Quotient > 2");
} else {
    puts("Condition is false");
}
```

&& сначала вычисляет левый операнд

b == 0 → **деления не будет**

|| не вычисляет правый операнд, если левый истинен

ВОПРОС

Битовые операции

```
unsigned x = 10U; // ...1010
unsigned y = 12U; // ...1100

x & y     // ?
x | y     // ?
x ^ y     // ?
x << 1    // ?
```

Знак	Смысл
&	И
|	ИЛИ
^	исключ. ИЛИ
~	инверсия
<< >>	сдвиги

Битовые операции

```
unsigned x = 10U; // ...1010
unsigned y = 12U; // ...1100

x & y     // 8     1000
x | y     // 14    1110
x ^ y     // 6     0110
x << 1    // 20  10100
```

Знак	Смысл
&	И
|	ИЛИ
^	исключ. ИЛИ
~	инверсия
<< >>	сдвиги

ВОПРОС

Типы литералов

```
1 << 40       // ?
1ULL << 40    // ?
```

Чем отличаются эти две строки?

Типы литералов

Целый литерал без суффикса получает первый подходящий тип: **int**, **long**, **long long**.

U	L	LL
unsigned	long	long long

```
1 << 40      // UB: сдвиг >= ширины int
1ULL << 40   // OK: unsigned long long
```

Преобразования типов

Получение значения **одного типа** из значения **другого** по правилам языка.

Неявное

```
double x = 5;
```

автоматически

Явное

```
(double)a / b
```

записываем (тип)выражение

Потеря данных

```
(int)3.9 → 3
```

дробная часть отброшена

Нужно для смешанных вычислений, передачи аргументов и присваивания.

Правило 1: всё меньше int → int

Перед вычислением **char**, **short** и **bool** превращаются в **int**.

`char` `short` `bool` → **`int`**

```
char a = 100, b = 100;
a + b           // 200, тип int
sizeof(a + b)  // == sizeof(int), а не 1
```

Правило 2: к старшему типу

Разные типы операндов → оба приводятся к **старшему**.

Целые `int` → `long` → `long long`

Вещественные `float` → `double` → `long double`
старше любых целых

Ранг одинаковый, знак разный → побеждает **unsigned**

```
5 / 2.0   // 2.5, double
1 + 2L    // 3, long
1 + 2u    // 3, unsigned int
```

ВОПРОС

Преобразования

```
double a = 5 / 2;              // ?
double b = (double)5 / 2;      // ?
int c = 3.9;                   // ?
```

Чему равны a, b и c?

Преобразования: как это работает

1. Выражение вычисляется **по типам операндов**
2. Результат приводится **к типу переменной**

```
double a = 5 / 2;          // 2.0: оба операнда int
double b = (double)5 / 2;  // 2.5: явное приведение
int c = 3.9;               // 3: неявное
```

Явное приведение задает преобразование в указанном месте.

ВОПРОС

Знаковое и беззнаковое

```
if (-1 < 1u)
    printf("да\n");
else
    printf("нет\n");
```

Что напечатает программа?

Напечатает «нет»

```
-1 < 1u                 // int и unsigned int
(unsigned)-1 < 1u       // -1 стал unsigned
UINT_MAX < 1u           // 0, ложь
```

Ранг одинаковый, знак разный → **unsigned** (правило 2)

–1 превращается в **UINT_MAX**: 4294967295 при 32-битном unsigned

Не сравнивайте знаковые и беззнаковые без явного приведения

ВОПРОС

Приоритет: ловушки

```
2 + 3 * 4          // ?
(2 + 3) * 4        // ?
x & 1 == 0         // ?
(x & 1) == 0       // ?
```

Как компилятор расставит скобки?

Приоритет операций

№	Операции
1	`[] () . -> x++ x--`
2	`sizeof (тип) &x *x -x ~ ! ++x --x`
3	`* / %`
4	`+ -`
5	`<< >>`
6	`< > <= >=`
7	`== !=`

№	Операции
8	`&`
9	`^`
10	`|`
11	`&&`
12	`||`
13	`? :`
14	`= += -= *= /= %=`
15	`,`

1 выполняется первым, **15** – последним. Строка 14 – все присваивания.

Приоритет: ловушки

```
2 + 3 * 4          // 14
(2 + 3) * 4        // 20
x & 1 == 0         // x & (1 == 0) !
(x & 1) == 0       // x четное
```

== связывает сильнее, чем **&**, поэтому в битовых проверках нужны скобки.

Виды поведения программы

Определенное

5 / 2 дает 2 – правило обязательно

Implementation-defined

Знаковость **char**: выбор документирован

Unspecified

Порядок **f()** и **g()** в f() + g() не задан и не документирован

Undefined

1 / 0: никаких требований

ВОПРОС

Знаковость char

```
int char_is_signed(void) {
  char c = (char)-1;
  return c < 0;
}
```

Что вернет функция: 1 или 0?

ПРИМЕР · IMPLEMENTATION-DEFINED

Поведение, зависящее от реализации

Implementation-defined: стандарт допускает варианты, реализация **документирует** выбор

Зависит от компилятора, платформы и флагов

Обе версии программы **корректны**

```
int char_is_signed(void) {
  char c = (char)-1;
  return c < 0;
}
```

GCC 14.1	Вывод
-fsigned-char	1
-funsigned-char	0

ВОПРОС

Две функции в одном выражении

```
int f(void) { printf("f "); return 1; }
int g(void) { printf("g "); return 2; }

int main(void) {
    int s = f() + g();
    printf("%d\n", s);
}
```

Что напечатает программа?

ПРИМЕР · UNSPECIFIED

Порядок вычисления операндов

```c
int f(void) { printf("f "); return 1; }
int g(void) { printf("g "); return 2; }

int main(void) {
    int s = f() + g();
    printf("%d\n", s);
}
```

Сумма всегда **3**

Кто вызовется первым, **f или g**, – не задано

```
f g 3
g f 3
```

Оба вывода корректны

Неопределенное поведение (UB)

Undefined behavior: стандарт не предъявляет к программе никаких требований.

Может случиться что угодно

Падение, неверный результат, неожиданный код.

Ваш кот может уронить люстру. Даже если у вас нет кота.

Примеры: деление на 0, знаковое переполнение, выход за границы массива

Компилятор оптимизирует, **считая, что UB не бывает**

Удачный запуск **ничего не доказывает**

ВОПРОС

Всегда ли x + 1 > x?

```
int greater(int x) {
    return x + 1 > x;
}
```

Что вернет greater(INT_MAX)?

ПРИМЕР · UNDEFINED BEHAVIOR

UB и оптимизация: x + 1 > x

```
int greater(int x) {
    return x + 1 > x;
}
```

Clang 18.1, -O2, x86-64:

```
greater:
    mov   eax, 1
    ret
```

Обычно результат **1**

При x == INT_MAX – **знаковое переполнение, UB**

Компилятор вправе считать, что UB нет, и **всегда вернуть 1**

ПРИМЕР · UNDEFINED BEHAVIOR

Функция, которую никто не вызывал

```
int main() {
  unsigned x = 0;
  while (x >= 0) {
    x = x + 1;
  }
}

void unreachable_function() {
  printf("You would never see this!\n");
}
```

x беззнаковый – цикл **бесконечный**

Побочных эффектов нет – **это UB**

Что выведет программа при -O0 и -Ofast?

Что запомнить

1. Стандарт – **общие правила** для программиста и компилятора
2. Размеры типов **не фиксированы**: проверяйте sizeof и <limits.h>
3. Тип выражения решают **операнды**: 5 / 2 == 2
4. Младший тип приводится к **старшему**: int + double → double
5. Сомневаетесь в приоритете – **ставьте скобки**
6. **UB** – не ошибка компиляции: программа может «работать» и быть неверной

Литература: язык C

«Язык программирования C», 2-е изд.

Б. Керниган, Д. Ритчи

C Primer Plus

С. Прата

Черновик стандарта C: N1256

Единственный справочник на контрольных – **учитесь с ним работать**

open-std.org/.../n1256.pdf

Литература: алгоритмы

«Грокаем алгоритмы»

А. Бхаргава

Algorithms Illuminated, части 1–4

Т. Рафгарден

«Алгоритмы. Построение и анализ»

Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн